

Curso _____
Sistemas Operativos

**UNIVERSIDAD
DE ANTIOQUIA**

Clase 18 - Semaforos (Parte 2)

Agenda

- Resumen clase anterior
- Contextualización
- Problema productor consumidor
- Problema de los lectores - Escritores
- Problema de los filósofos
- Implementación de Zemáforos
- Conclusiones

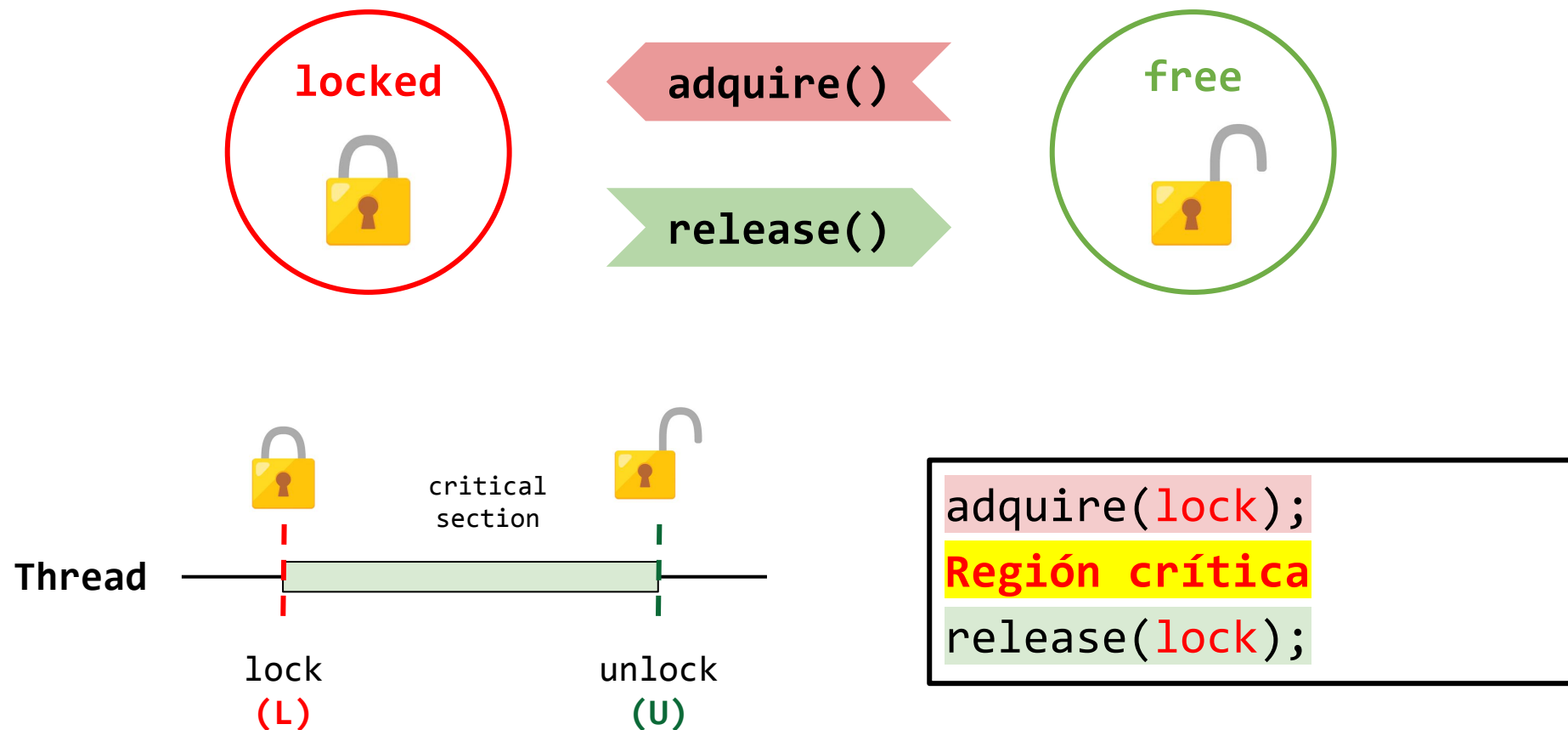
Agenda

- **Resumen clase anterior**
- Contextualización
- Problema productor consumidor
- Problema de los lectores - Escritores
- Problema de los filósofos
- Implementación de Zemáforos
- Conclusiones

Resumen clase anterior

Locks

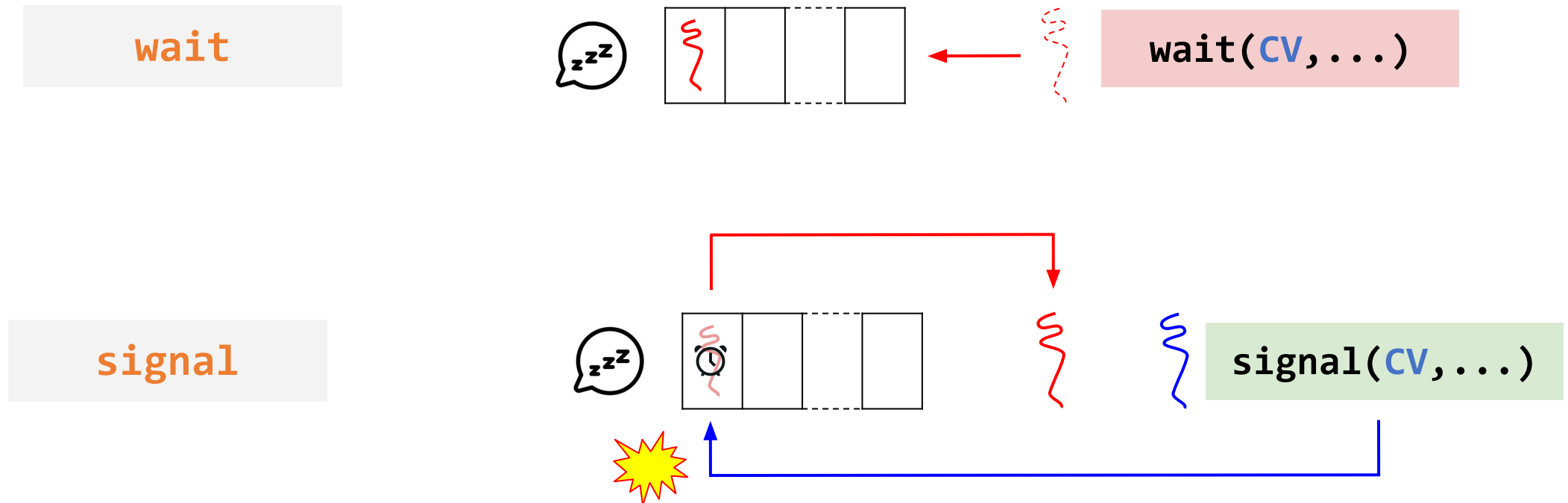
- Un lock es un objeto de memoria que proporciona exclusión mutua.



Resumen clase anterior

Variables de condición (CV)

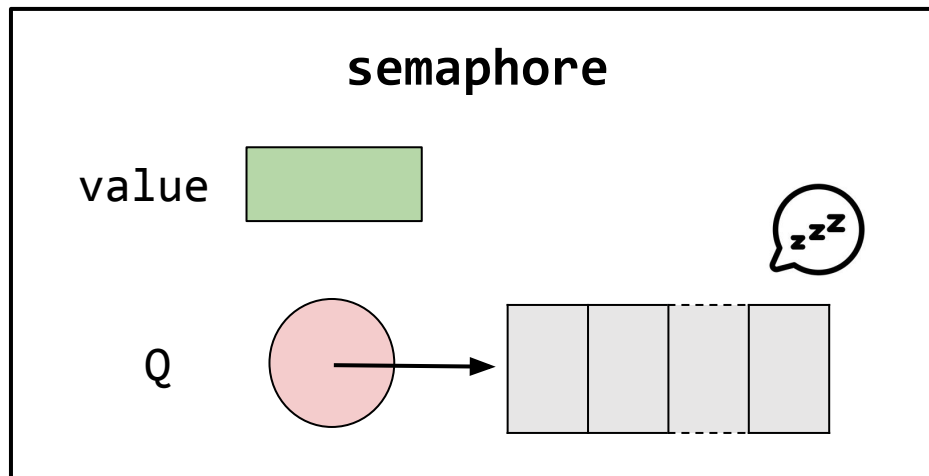
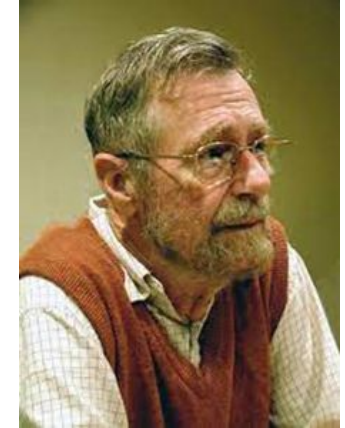
- Primitiva de sincronización inventada por Dijkstra en 1968.
- **Objeto con un valor entero** no negativo (asociado al estado).



Resumen clase anterior

Semáforo

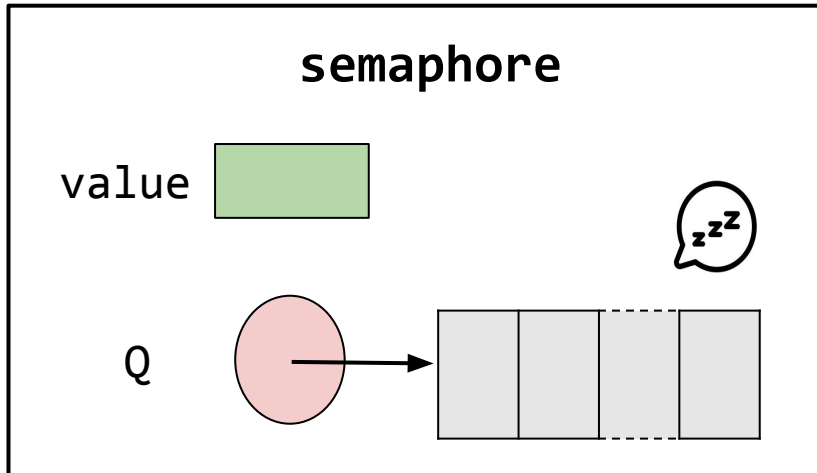
- Objeto con un valor entero no negativo (asociado al estado).
- Cola de hilos que se encuentran dormidos.



```
typedef struct {  
    int value;  
    struct process *Q;  
} sem_t;
```

Resumen clase anterior

Semáforo



```
typedef struct {  
    int value;  
    struct process *Q;  
} sem_t;
```

init

```
sem_init(sem_t *s, int value) {  
    s->value = initval  
}
```

wait

```
void sem_wait(sem_t *s) { // Must be executed atomically  
    s->value--;  
    if (S->value < 0) {  
        add this process to s->Q;  
        block();  
    }  
}
```

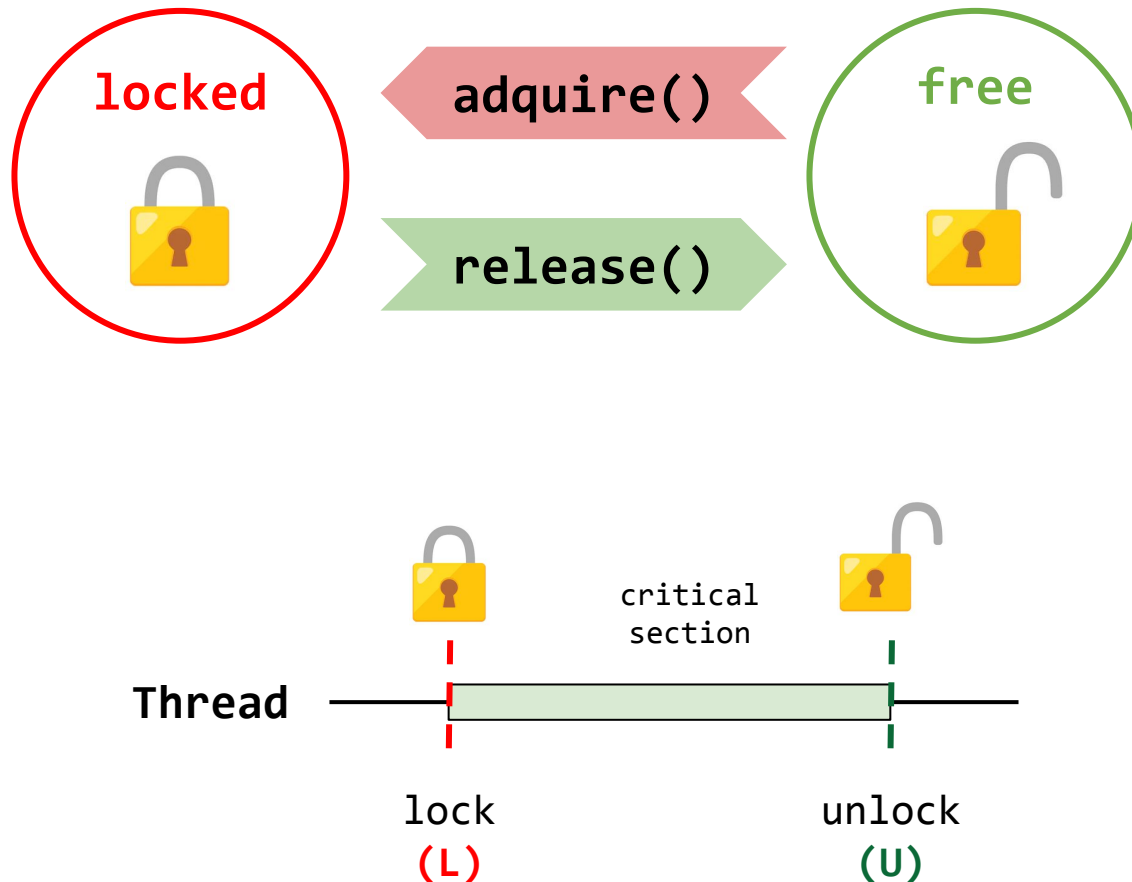
post

```
void sem_post(sem_t *s) { // Must be executed atomically  
    s->value++;  
    if (s->value <= 0) {  
        remove a process P from s->Q;  
        wakeup(P);  
    }  
}
```

Resumen clase anterior

Código de implementación del lock empleando semáforos

Es posible implementar **locks** empleando semáforos binarios

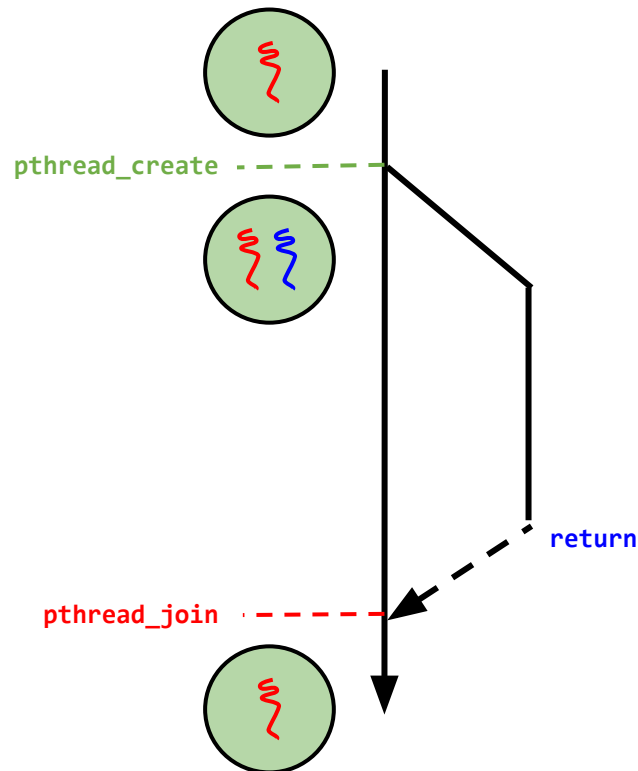


```
typedef struct __lock_t {  
    // whatever data structs you need goes here  
    sem_t value;  
} lock_t;  
  
void init(lock_t *lock) {  
    // init code goes here  
    sem_init(&lock->value, 0, 1);  
}  
  
void acquire(lock_t *lock) {  
    // lock acquire code goes here  
    sem_wait(&lock->value);  
}  
  
void release(lock_t *lock) {  
    // lock release code goes here  
    sem_post(&lock->value);  
}
```


Resumen clase anterior

Implementación del join usando semáforos

La implementación del **join** es más simple con **semáforos** que con **variables de condición (CV)**.



```
sem_t s;  
  
void *child(void *arg) {  
    printf("child\n");  
    sem_post(&s); // increment  
}  
  
int main(int argc, char *argv[]) {  
    sem_init(&s, 0, 0);  
    pthread_t c;  
    printf("parent: begin\n");  
    pthread_create(&c, NULL, child, NULL);  
    sem_wait(&s); // decrement, wait for counter == 0  
    printf("parent: end\n");  
}
```

Agenda

- Resumen clase anterior
- **Contextualización**
- Problema productor consumidor
- Problema de los lectores - Escritores
- Problema de los filósofos
- Implementación de Zemáforos
- Conclusiones

Contextualización

Pregunta clave

¿Cómo usar semáforos?

- ¿Cómo implementar el problema del productor consumidor usando **semáforos**?
- ¿Qué otros problemas pueden ser resueltos empleando **semáforos**?



Agenda

- Resumen clase anterior
- Contextualización
- **Problema productor consumidor**
- Problema de los lectores - Escritores
- Problema de los filósofos
- Implementación de Zemáforos
- Conclusiones

Problema productor-consumidor

- También conocido como el problema del buffer limitado (Bounded Buffer Problem)
- El productor y el consumidor se ejecutan a tasas diferentes:
 - No hay serialización de uno detrás de otro.
 - Las tareas son independientes.

Productor

P1

P2

...

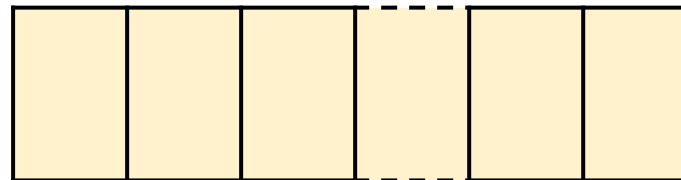
Pi

...

Pn

Pone elementos en el buffer.

Buffer



Conjunto de elementos compartidos

Consumidor

C1

C2

...

Cj

...

Cn

Saca elementos del buffer



Problema productor-consumidor

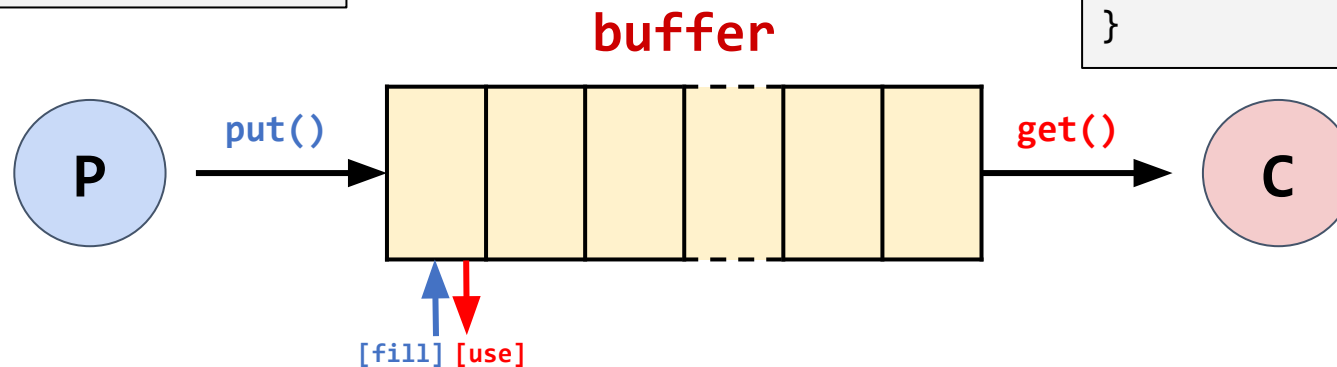
Rutinas put y get

put()

```
void put(int value) {  
    buffer[fill] = value; // Line F1  
    fill = (fill + 1)%MAX; // Line F2  
}
```

get()

```
int get() {  
    int tmp = buffer[use]; // Line G1  
    use = (use + 1)%MAX; // Line G2  
    return tmp;  
}
```



```
int buffer[MAX];  
int fill = 0;  
int use = 0;
```

Problema productor-consumidor

Implementación 1

Se emplean dos semáforos:

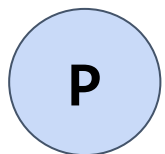
- **empty**: Buffer vacío
- **full**: Buffer lleno

```
sem_t empty = 0;
sem_t full = 0;

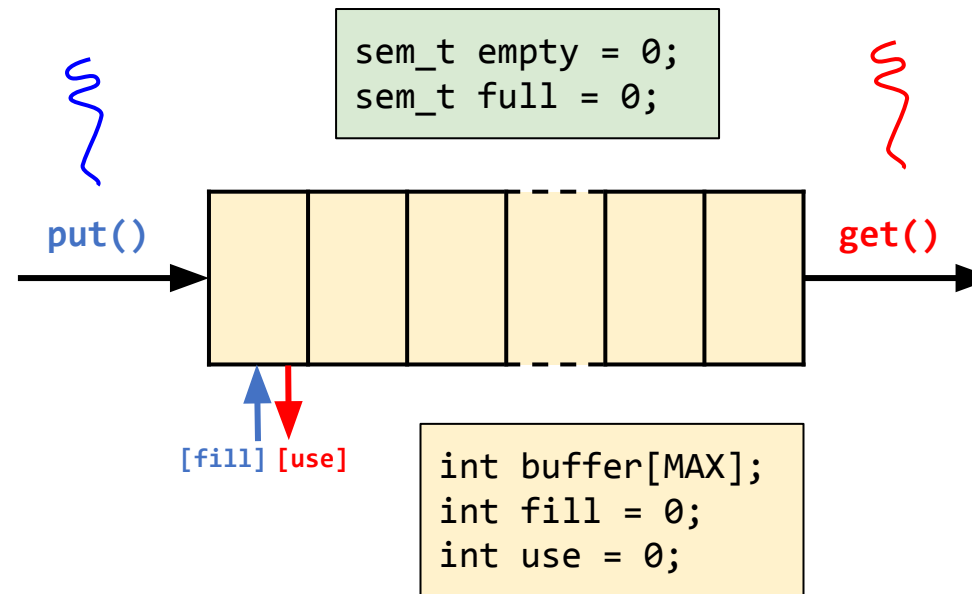
int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX); // MAX are empty
    sem_init(&full, 0, 0);    // 0 are full
    // ...
}
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty); // Line P1
        put(i);           // Line P2
        sem_post(&full);  // Line P3
    }
}
```

Productor → put()

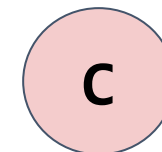


Espera a que el buffer se vacíe para colocar datos en este



```
void *consumer(void *arg) {
    int tmp = 0;
    while (tmp != -1) {
        sem_wait(&full); // Line C1
        tmp = get();     // Line C2
        sem_post(&empty); // Line C3
        printf("%d\n", tmp);
    }
}
```

Consumidor → get()



Espera a que el buffer se vuelva a llenar para usarlo

Problema productor-consumidor

Implementación 1 - Caso 1

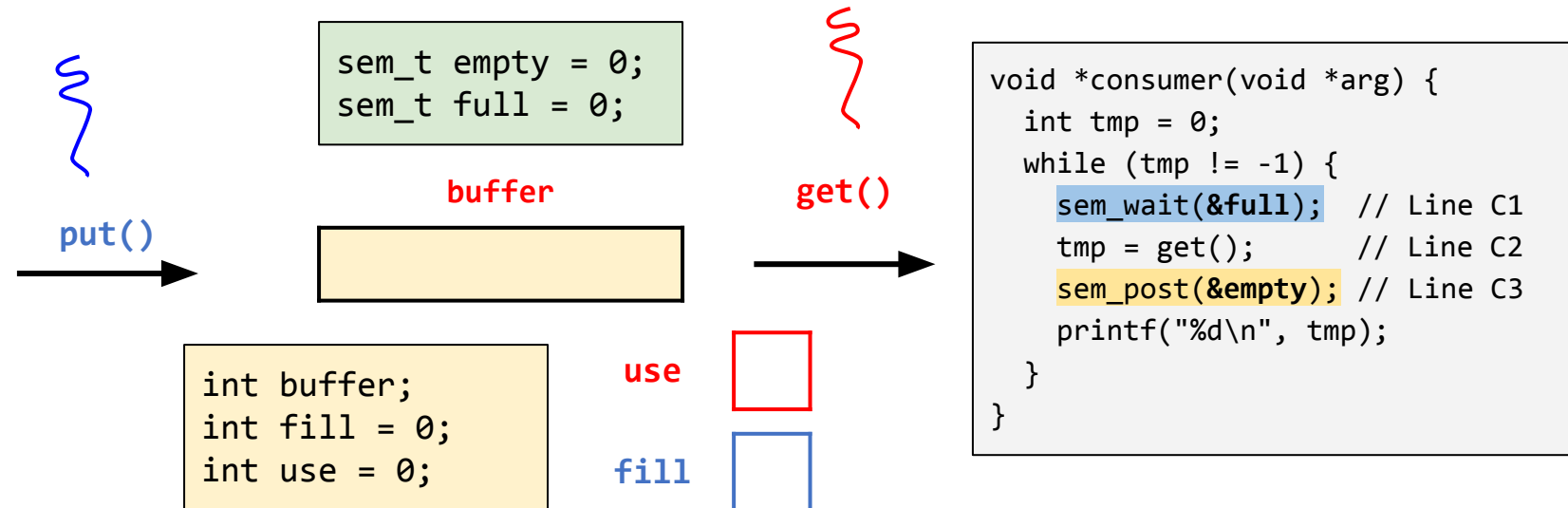
Caso 1: Single buffer

- **Dos hilos:**
 - Productor
 - Consumidor
- **Single buffer:** MAX = 1

```
sem_t empty = 0;
sem_t full = 0;

int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, 1); // 1 is empty
    sem_init(&full, 0, 0);  // 0 is full
    // ...
}
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty); // Line P1
        put(i);           // Line P2
        sem_post(&full);  // Line P3
    }
}
```



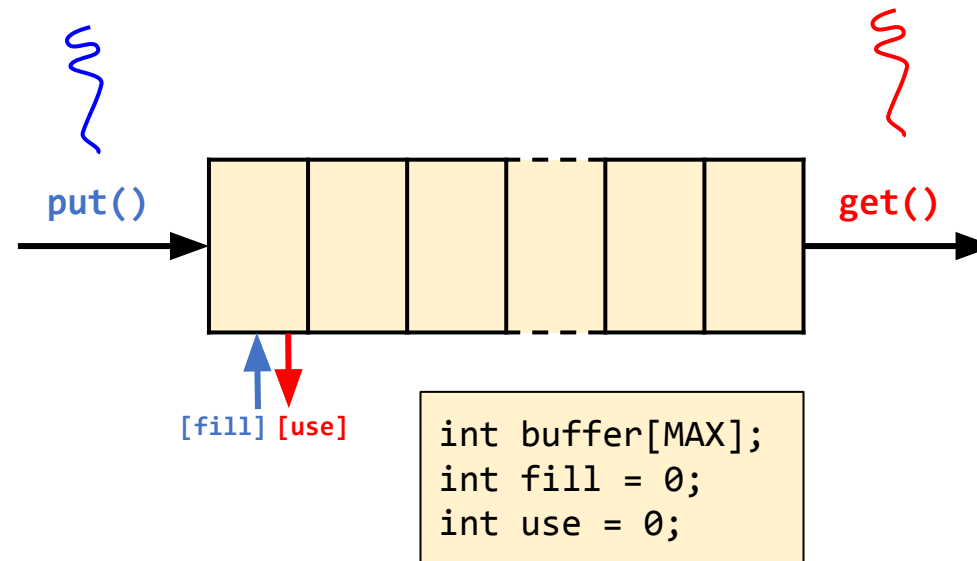
Problema productor-consumidor

Implementación 1 - Caso 2

Caso 2: Buffer circular

- El buffer comparte varios elementos ($MAX > 1$) entre productores y consumidores.

```
void *producer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        sem_wait(&empty); // Line P1  
        put(i);           // Line P2  
        sem_post(&full);  // Line P3  
    }  
}
```



```
void *consumer(void *arg) {  
    int tmp = 0;  
    while (tmp != -1) {  
        sem_wait(&full); // Line C1  
        tmp = get();     // Line C2  
        sem_post(&empty); // Line C3  
        printf("%d\n", tmp);  
    }  
}
```

Problema productor-consumidor

Implementación 1 - Caso 2

Caso 2: Buffer circular

Requiere dos semáforos:

- **empty**: Que se inicializa a **MAX** (**MAX** espacios vacíos (empty buffers) de modo que el productor puede ejecutarse **MAX** veces primero).
- **full**: Que se inicializa a **0** (**0** espacios llenos (full buffers)).

```
sem_t empty = 0;
sem_t full = 0;

int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX); // MAX are empty
    sem_init(&full, 0, 0);    // 0 are full
    // ...
}
```

Problema productor-consumidor

Implementación 1 - Caso 2

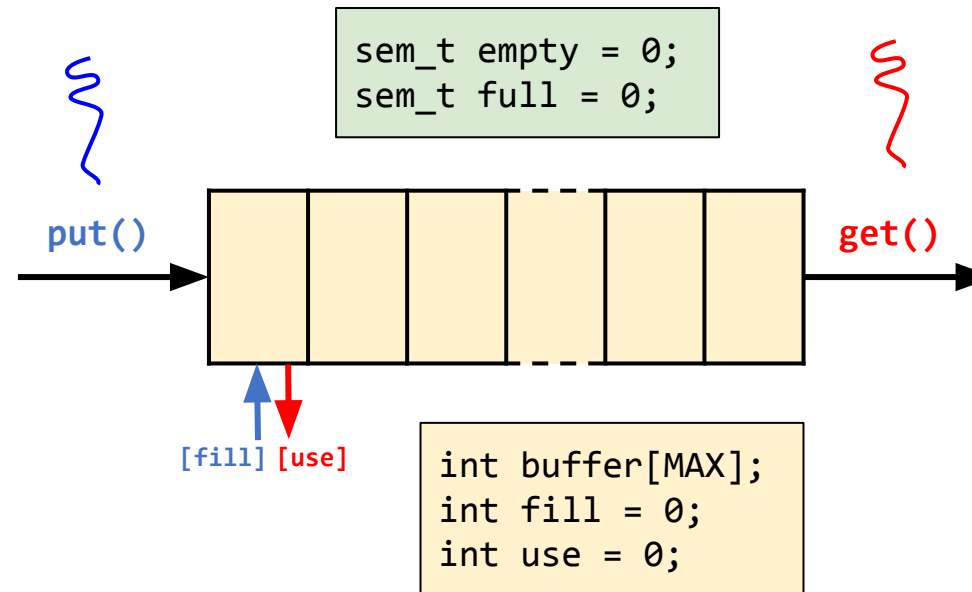
Caso 2: Buffer circular

- El buffer comparte varios elementos ($MAX > 1$) entre productores y consumidores.

```
sem_t empty = 0;
sem_t full = 0;

int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX); // MAX are empty
    sem_init(&full, 0, 0);    // 0 are full
    // ...
}
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty); // Line P1
        put(i);           // Line P2
        sem_post(&full);  // Line P3
    }
}
```



```
void *consumer(void *arg) {
    int tmp = 0;
    while (tmp != -1) {
        sem_wait(&full); // Line C1
        tmp = get();     // Line C2
        sem_post(&empty); // Line C3
        printf("%d\n", tmp);
    }
}
```

Problema productor-consumidor

Implementación 1 - Caso 2

Caso 2: Buffer circular

Problema: Llenar el buffer e incrementar el contador es una sección crítica.

- Si MAX es mayor que 1 ($MAX > 1$):
 - **Race condición** en función **put** (línea F1).
 - El dato antiguo será sobrescrito.



```
sem_t empty = MAX;
sem_t full = 0;
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty); // Line P1
        put(i);           // Line P2
        sem_post(&full);  // Line P3
    }
}
```

```
void put(int value) {
    buffer[fill] = value; // Line F1
    fill = (fill + 1)%MAX; // Line F2
}
```

Hilos		Semáforos		Comentarios
Pa	Pb	empty	full	
---	---	MAX	0	Valores iniciales: [fill = 0]
P1		MAX - 1	0	P1 -> wait(empty) [fill = 0]
P2		MAX - 1	0	-> put() [fill = 0]
F1		MAX - 1	0	Pa agrega elemento en la posición 0 del buffer
		MAX - 1	0	Interrupción: Pa → Pb
	P1	MAX - 2	0	P2 -> wait(empty)
	P2	MAX - 2	0	-> put() [fill = 0]
	F1	MAX - 2	0	Pb agrega elemento en la posición 0 del buffer (Sobrescritura: ouch...)
	F2	MAX - 2	0	[fill = 1]

Traza para dos productores: Pa y Pb.



Problema productor-consumidor

Implementación 1 - Caso 2

Caso 2: Buffer circular

Problema: Race condition

Solución: Agregar exclusión mutua para proteger la sección crítica (Llenar el buffer e incrementar el índice en el buffer)

```
int main(int argc, char *argv[]) {  
    // ...  
    sem_init(&empty, 0, MAX); // MAX are empty  
    sem_init(&full, 0, 0);     // 0 are full  
    // ...  
}
```

```
int main(int argc, char *argv[]) {  
    // ...  
    sem_init(&empty, 0, MAX); // MAX are empty  
    sem_init(&full, 0, 0);     // 0 are full  
    sem_init(&mutex, 0, 1);    // mutex = 1  
    // ...  
}
```

Problema productor-consumidor

Implementación 1 - Caso 2

Caso 2: Buffer circular

Solución: Agregar exclusión mutua para proteger la sección crítica (Llenar el buffer e incrementar el índice en el buffer)

Problema: Race condition

```
void *producer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        sem_wait(&empty); // Line P1  
        put(i);           // Line P2  
        sem_post(&full);  // Line P3  
    }  
}
```

```
void *consumer(void *arg) {  
    int tmp = 0;  
    while (tmp != -1) {  
        sem_wait(&full); // Line C1  
        tmp = get();     // Line C2  
        sem_post(&empty); // Line C3  
        printf("%d\n", tmp);  
    }  
}
```

```
sem_t empty = MAX;  
sem_t full = 0;  
sem_t mutex = 1;
```

```
void *producer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        sem_wait(&mutex); // line P0 (NEW LINE)  
        sem_wait(&empty); // Line P1  
        put(i);           // Line P2  
        sem_post(&full);  // Line P3  
        sem_post(&mutex); // line P4 (NEW LINE)  
    }  
}
```

```
void *consumer(void *arg) {  
    int tmp = 0;  
    while (tmp != -1) {  
        sem_wait(&mutex); // line c0 (NEW LINE)  
        sem_wait(&full);  // Line C1  
        tmp = get();      // Line C2  
        sem_post(&empty); // Line C3  
        sem_post(&mutex); // line c4 (NEW LINE)  
        printf("%d\n", tmp);  
    }  
}
```

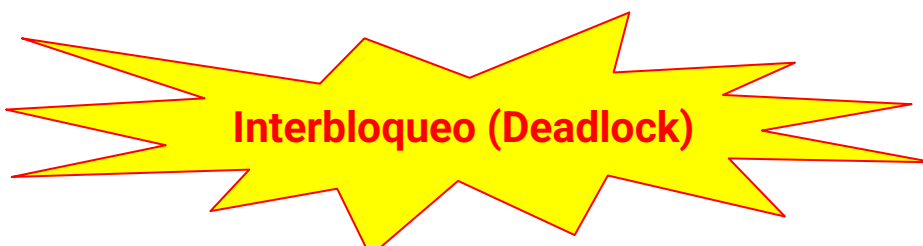
Problema productor-consumidor

Implementación 1 - Caso 2

Análisis de la solución que agregar exclusión mutua

Suponga que se tienen dos hilos uno productor y uno consumidor.

- El consumidor adquiere el **mutex**. (line C0).
- El consumidor llama **sem_wait()** en el semáforo full. (line C1)
- El consumidor está bloqueado y libera la CPU.
- El consumidor aún posee el **mutex**.
- El productor llama **sem_wait()** en semáforo **mutex** (line P0)
- El productor está bloqueado.



Interbloqueo (Deadlock)

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex); // line P0 (NEW LINE)
        sem_wait(&empty); // Line P1
        put(i);           // Line P2
        sem_post(&full);  // Line P3
        sem_post(&mutex); // line P4 (NEW LINE)
    }
}
```

```
void *consumer(void *arg) {
    int tmp = 0;
    while (tmp != -1) {
        sem_wait(&mutex); // line c0 (NEW LINE)
        sem_wait(&full);  // Line C1
        tmp = get();      // Line C2
        sem_post(&empty); // Line C3
        sem_post(&mutex); // line c4 (NEW LINE)
        printf("%d\n", tmp);
    }
}
```

Problema productor-consumidor

Implementación 1 - Caso 2

Análisis de la solución que agregar exclusión mutua

P

```

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex); // line P0 (NEW LINE)
        sem_wait(&empty); // Line P1
        put(i);           // Line P2
        sem_post(&full);   // Line P3
        sem_post(&mutex);  // line P4 (NEW LINE)
    }
}
  
```

C

```

void *consumer(void *arg) {
    int tmp = 0;
    while (tmp != -1) {
        sem_wait(&mutex); // line c0 (NEW LINE)
        sem_wait(&full);   // Line C1
        tmp = get();       // Line C2
        sem_post(&empty);  // Line C3
        sem_post(&mutex);  // line c4 (NEW LINE)
        printf("%d\n", tmp);
    }
}
  
```

Hilos		Semáforos			Comentarios
P	C	empty	full	mutex	
---	---	MAX	0	1	Inicialización
	C0	MAX	0	0	C adquiere el mutex: C0 -> wait(mutex)
	C1	MAX - 1	-1	0	C se bloquea: C1 -> wait(empty)
		MAX - 1	0	0	Interrupción: C → P
P0		MAX - 1	0	-1	P se bloquea: P0 -> wait(mutex)

Interbloqueo (Deadlock)

Problema productor-consumidor

Implementación 1 - Caso 2

Caso 2: Buffer circular

Problema: Race condition

Solución: Agregar exclusión mutua para proteger la sección crítica (Llenar el buffer e incrementar el índice en el buffer)

La inicialización de los semáforos no cambia

```
int main(int argc, char *argv[]) {  
    // ...  
    sem_init(&empty, 0, MAX); // MAX are empty  
    sem_init(&full, 0, 0);     // 0 are full  
    sem_init(&mutex, 0, 1);    // mutex = 1  
    // ...  
}
```

Problema productor-consumidor

Implementación 1 - Caso 2

Caso 2: Buffer circular

Problema: Deadlock

Solución: Reducir el ámbito del lock (reubicar la sección encargada de la exclusión mutua)

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex); // line P0 (NEW LINE)
        sem_wait(&empty); // Line P1
        put(i);           // Line P2
        sem_post(&full);  // Line P3
        sem_post(&mutex); // line P4 (NEW LINE)
    }
}
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty); // Line P1
        sem_wait(&mutex); // line P2 (MUTEX HERE)
        put(i);           // Line P3
        sem_post(&mutex); // line P4 (MUTEX HERE)
        sem_post(&full);  // Line P4
    }
}
```

Problema productor-consumidor

Implementación 1 - Caso 2

Caso 2: Buffer circular

Problema: Deadlock

Solución: Reducir el ámbito del lock (reubicar la sección encargada de la exclusión mutua)

```
void *consumer(void *arg) {
    int tmp = 0;
    while (tmp != -1) {
        sem_wait(&mutex); // line C0 (NEW LINE)
        sem_wait(&full); // Line C1
        tmp = get(); // Line C2
        sem_post(&empty); // Line C3
        sem_post(&mutex); // line C4 (NEW LINE)
        printf("%d\n", tmp);
    }
}
```

```
void *consumer(void *arg) {
    int tmp = 0;
    while (tmp != -1) {
        sem_wait(&full); // Line C1
        sem_wait(&mutex); // line C2 (MUTEX HERE)
        tmp = get(); // Line C3
        sem_post(&mutex); // line C4 (MUTEX HERE)
        sem_post(&empty); // Line C5
        printf("%d\n", tmp);
    }
}
```

Problema productor-consumidor

Implementación 1 - Caso 2

Análisis: Reubicación del mutex

```

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty); // Line P1
        sem_wait(&mutex); // line P2
        put(i);           // Line P3
        sem_post(&mutex); // line P4
        sem_post(&full);  // Line P5
    }
}

```

```

void *consumer(void *arg) {
    int tmp = 0;
    while (tmp != -1) {
        sem_wait(&full); // Line C1
        sem_wait(&mutex); // line C2
        tmp = get();      // Line C3
        sem_post(&mutex); // line C4
        sem_post(&empty); // Line C5
        printf("%d\n", tmp);
    }
}

```

Hilos		Semáforos			Comentarios
P	C	empty	full	mutex	
---	---	MAX	0	1	Inicialización
	C1	MAX	-1	1	C se bloquea: C1 -> wait(full)
		MAX	-1	1	Interrupción: C → P
P1		MAX - 1	-1	1	P1 -> wait(empty)
P2		MAX - 1	-1	0	P adquiere el mutex: P2 -> wait(mutex)
P3		MAX - 1	-1	0	P entra en la región crítica
P4		MAX - 1	-1	1	P libera el mutex: P4 -> post(mutex)
P5		MAX - 1	0	1	P4 -> post(full) C se despierta
		MAX - 1	0	1	Interrupción: P → C
	C2	MAX - 1	0	0	C adquiere el mutex: C2 -> wait(mutex)
	C3	MAX - 1	0	0	C entra en la región crítica
	C4	MAX - 1	0	1	P libera el mutex: C4 -> post(mutex)
	C5	MAX	0	1	C5 -> post(empty)

Problema productor-consumidor

Implementación definitiva

```
sem_t empty;
sem_t full;
sem_t mutex;

int loops = 50; // Variable global

int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX); // MAX are empty
    sem_init(&full, 0, 0);    // 0 are full
    sem_init(&mutex, 0, 1);   // mutex = 1
    // ...
    pthread_t C, P;
    pthread_create(P, NULL, producer, NULL);
    pthread_create(C, NULL, consumer, NULL);
    // ...
    return 0;
}
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty); // Line P1
        sem_wait(&mutex); // line P2
        put(i);           // Line P3
        sem_post(&mutex); // line P4
        sem_post(&full);  // Line P5
    }
}
```

```
void *consumer(void *arg) {
    int tmp = 0;
    while (tmp != -1) {
        sem_wait(&full); // Line C1
        sem_wait(&mutex); // line C2
        tmp = get();     // Line C3
        sem_post(&mutex); // line C4
        sem_post(&empty); // Line C5
        printf("%d\n", tmp);
    }
}
```

Agenda

- Resumen clase anterior
- Contextualización
- Problema productor consumidor
- **Problema de los lectores - Escritores**
- Problema de los filósofos
- Implementación de Zemáforos
- Conclusiones

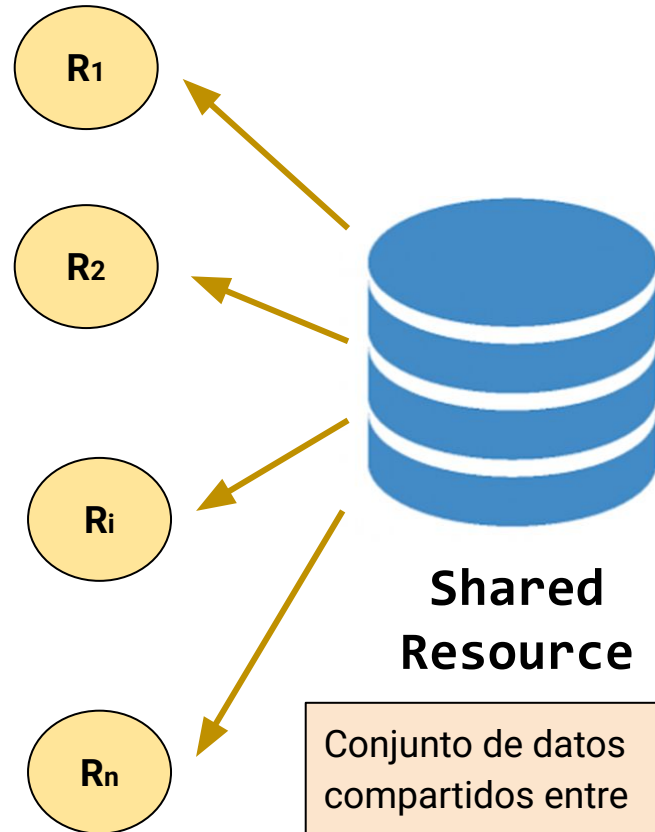
Problema de los lectores - Escritores

Problema en el cual se tienen un número de operaciones concurrentes sobre los datos (compartidos) que solo son lecturas y escrituras.

Readers (lectores)



Solo leen el conjunto de datos (no realizan ninguna actualización).

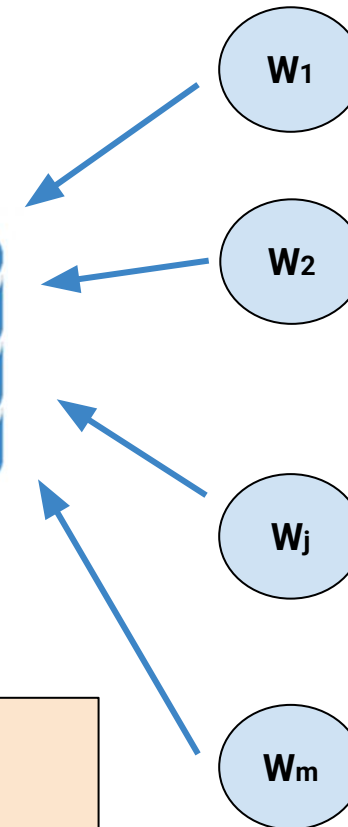


Conjunto de datos compartidos entre procesos concurrentes.

Writers (Escritores)



Pueden leer y escribir (por lo tanto pueden cambiar el estado de los datos)

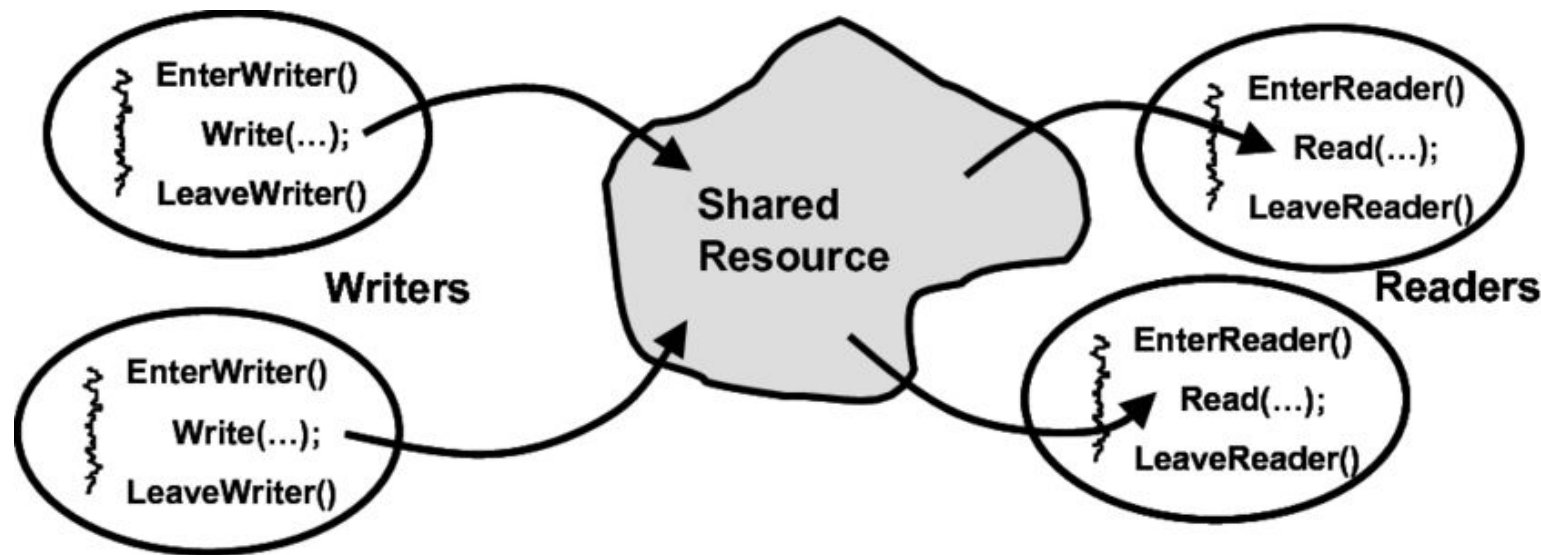


Problema de los lectores - Escritores

Sobre el problema

Las operaciones de lectura y escritura sobre los datos se ejecutan de manera concurrente teniendo en cuenta las siguientes restricciones:

- **Escritura:**
 - Solo un escritor puede acceder a los datos compartidos a la vez.
 - Cambia los datos (acceso a la región crítica)
- **Lectura:**
 - Solo lee los datos.
 - Es posible realizar varias lecturas de manera simultánea siempre y cuando no hayan escrituras realizándose.



Problema de los lectores - Escritores

Reader/Writer Lock (rwlock_t)

- Tipo especial de lock implementado para operaciones de lectura y escritura.
- Se puede implementar con semáforos:
 - **readers**: Número de hilos de lectura que se encuentran actualmente en la estructura de datos.
 - **writelock**: Lock de escritura.
 - **lock**: Lock de lectura.

```
typedef _struc_t rwlock_t {  
    sem_t lock;           // binary semaphore (basic lock)  
    sem_t writelock ;     // used to allow ONE writer or MANY readers  
    int readers;          // count of readers reading in critical section  
} rwlock_t ;
```

Problema de los lectores - Escritores

Operaciones estructura Reader/Writer Lock (rwlock_t)

Inicialización

- **rwlock_init**: Inicializa la estructura **rwlock_t**

```
void rwlock_init(rwlock_t *rw) {  
    rw->readers = 0;  
    sem_init(&rw->lock, 0, 1);  
    sem_init(&rw->writelock, 0, 1);  
}
```

Operaciones de sincronización básicas para actualización de datos

- **rwlock_acquire_writelock**: Permite al escritor adquirir el **writelock**

```
void rwlock_acquire_writelock(rwlock_t *rw) {  
    sem_wait(&rw->writelock);  
}
```

- **rwlock_release_writelock**: Libera el **writelock**

```
void rwlock_release_writelock(rwlock_t *rw) {  
    sem_post(&rw->writelock);  
}
```

Estas operaciones usan el semáforo **writelock** internamente para asegurar que solo un escritor puede ingresar a la región crítica para actualizar los datos.

Problema de los lectores - Escritores

Operaciones estructura Reader/Writer Lock (rwlock_t)

Operaciones de sincronización básicas la lectura de datos

- **rwlock_acquire_readlock**: Permite al lector adquirir el **lock**

```
void rwlock_acquire_readlock(rwlock_t *rw) {  
    sem_wait(&rw->lock);  
    rw->readers++;  
    if (rw->readers == 1) // first reader gets writelock  
        sem_wait(&rw->writelock);  
    sem_post(&rw->lock);  
}
```

- **rwlock_release_readlock**: Libera el **Lock**

```
void rwlock_release_readlock(rwlock_t *rw) {  
    sem_wait(&rw->lock);  
    rw->readers--;  
    if (rw->readers == 0) // last reader lets it go  
        sem_post(&rw->writelock);  
    sem_post(&rw->lock);  
}
```

Problema de los lectores - Escritores

Resumen estructura y operaciones rwlock_t

```
typedef _struc_t rwlock_t {
    sem_t lock;           // binary semaphore (basic lock)
    sem_t writelock;      // used to allow ONE writer or MANY
                          // readers
    int readers;          // count of readers reading in
                          // critical section
} rwlock_t ;

void rwlock_init(rwlock_t *rw) {
    rw->readers = 0;
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->writelock, 0, 1);
}
```

```
void rwlock_acquire_writelock(rwlock_t *rw) {
    sem_wait(&rw->writelock);
}

void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->writelock);
}

void rwlock_acquire_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers++;
    if (rw->readers == 1) // first reader gets writelock
        sem_wait(&rw->writelock);
    sem_post(&rw->lock);
}

void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers--;
    if (rw->readers == 0) // last reader lets it go
        sem_post(&rw->writelock);
    sem_post(&rw->lock);
}
```

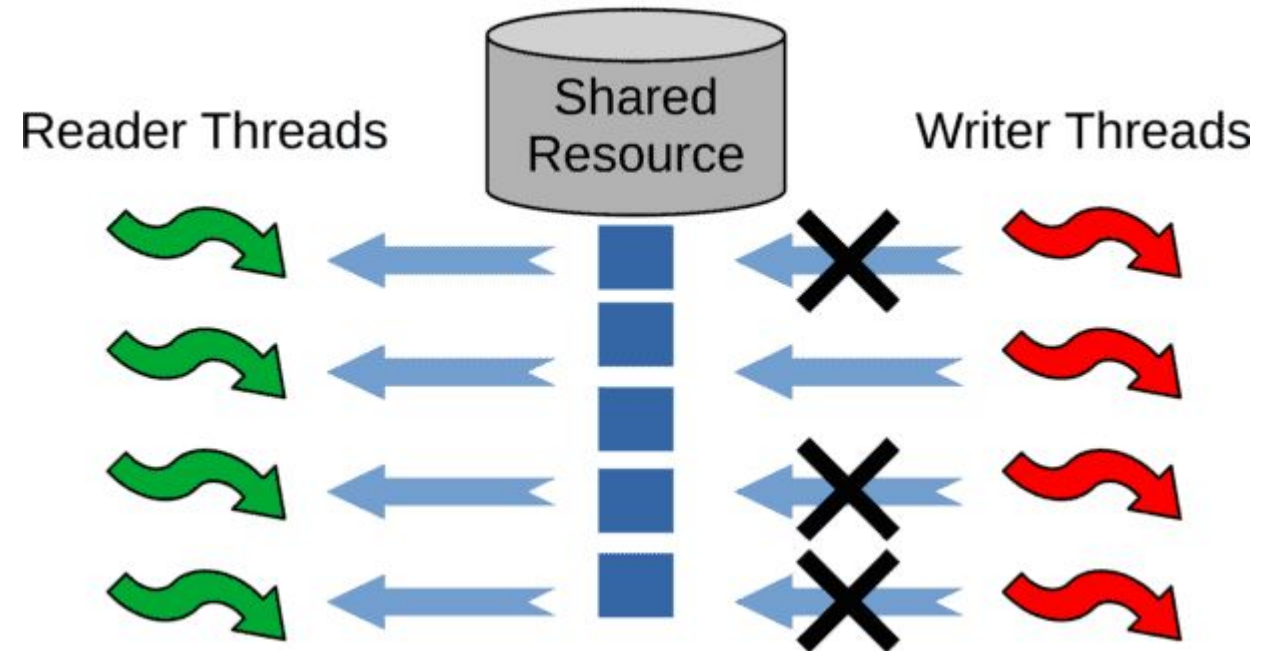
Problema de los lectores - Escritores

Reader/writer lock

- Solo un **writer** (escritor) puede entrar a la **región crítica**. (este writer es el único que adquiere el **lock de escritura** (`rw->writelock`))

```
void rwlock_acquire_writelock(rwlock_t *rw) {  
    sem_wait(&rw->writelock);  
}
```

```
void rwlock_release_writelock(rwlock_t *rw) {  
    sem_post(&rw->writelock);  
}
```



Problema de los lectores - Escritores

Reader/writer lock

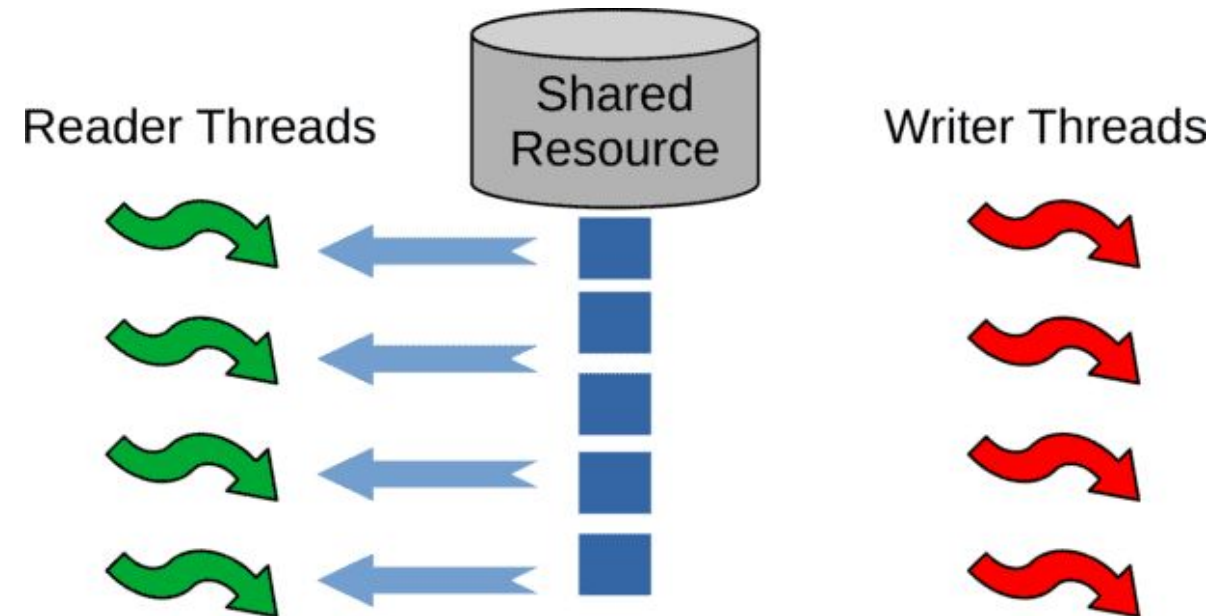
- Una vez que un lector ha adquirido el **write lock** (`rw->lock`)
 - Se permite que más lectores puedan realizar la lectura (del recurso compartido).
 - Cualquier escritor debe esperar a que todos los lectores finalicen.

El primer lector adquiere el **write lock**. (Open)

```
void rwlock_acquire_readlock(rwlock_t *rw) {  
    sem_wait(&rw->lock);  
    rw->readers++;  
    if (rw->readers == 1) // first reader gets writelock  
        sem_wait(&rw->writelock);  
    sem_post(&rw->lock);  
}
```

El último lector libera el **write lock** (Close)

```
void rwlock_release_readlock(rwlock_t *rw) {  
    sem_wait(&rw->lock);  
    rw->readers--;  
    if (rw->readers == 0) // last reader lets it go  
        sem_post(&rw->writelock);  
    sem_post(&rw->lock);  
}
```



Problema de los lectores - Escritores

Problema de la equidad

- Muchos lectores pueden fácilmente denegar el acceso al escritor (**starvation**).
- Existen soluciones más sofisticadas a este problema. Por ejemplo: Evitar el acceso de nuevos lectores cuando haya un escritor en espera

Para profundizar más sobre esto y otras cosas puede consultar el siguiente [link](#)).



Agenda

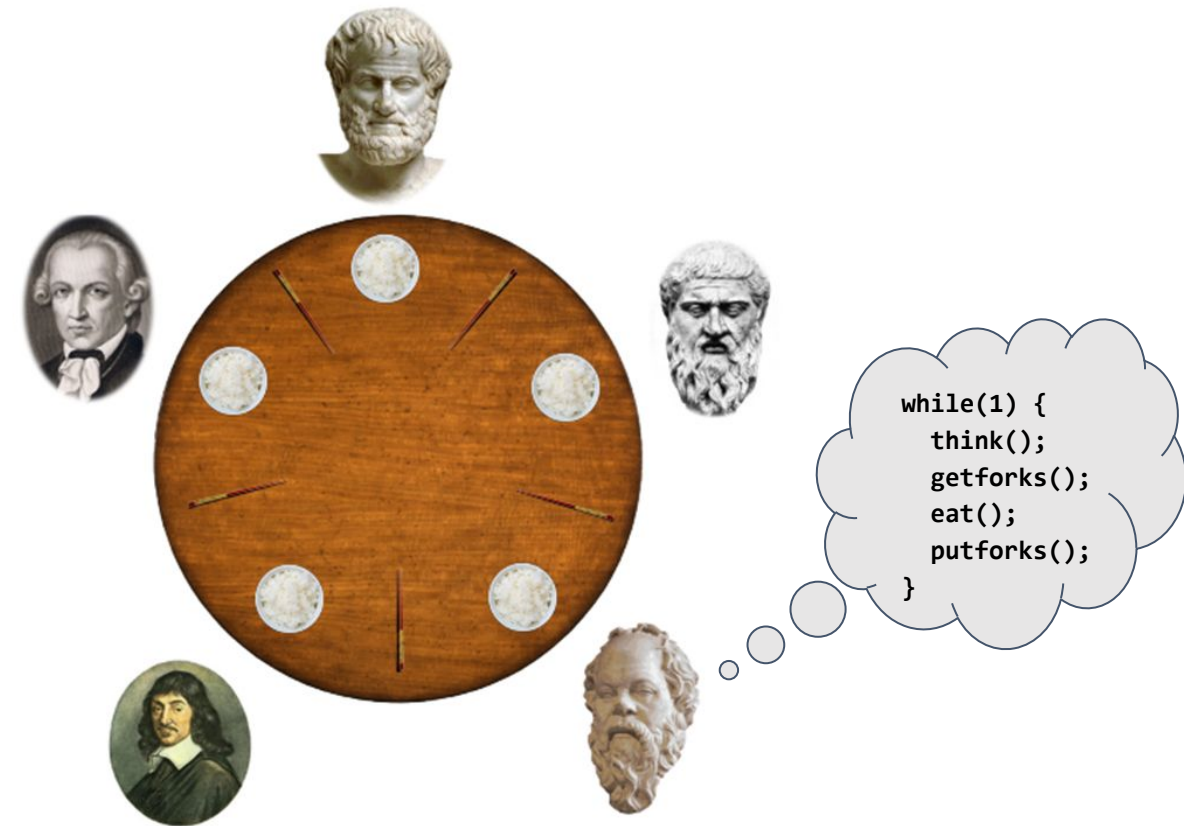
- Resumen clase anterior
- Contextualización
- Problema productor consumidor
- Problema de los lectores - Escritores
- **Problema de los filósofos**
- Implementación de Zemáforos
- Conclusiones

Problema de los filósofos

Enunciado del problema (Pienso luego... como comida)

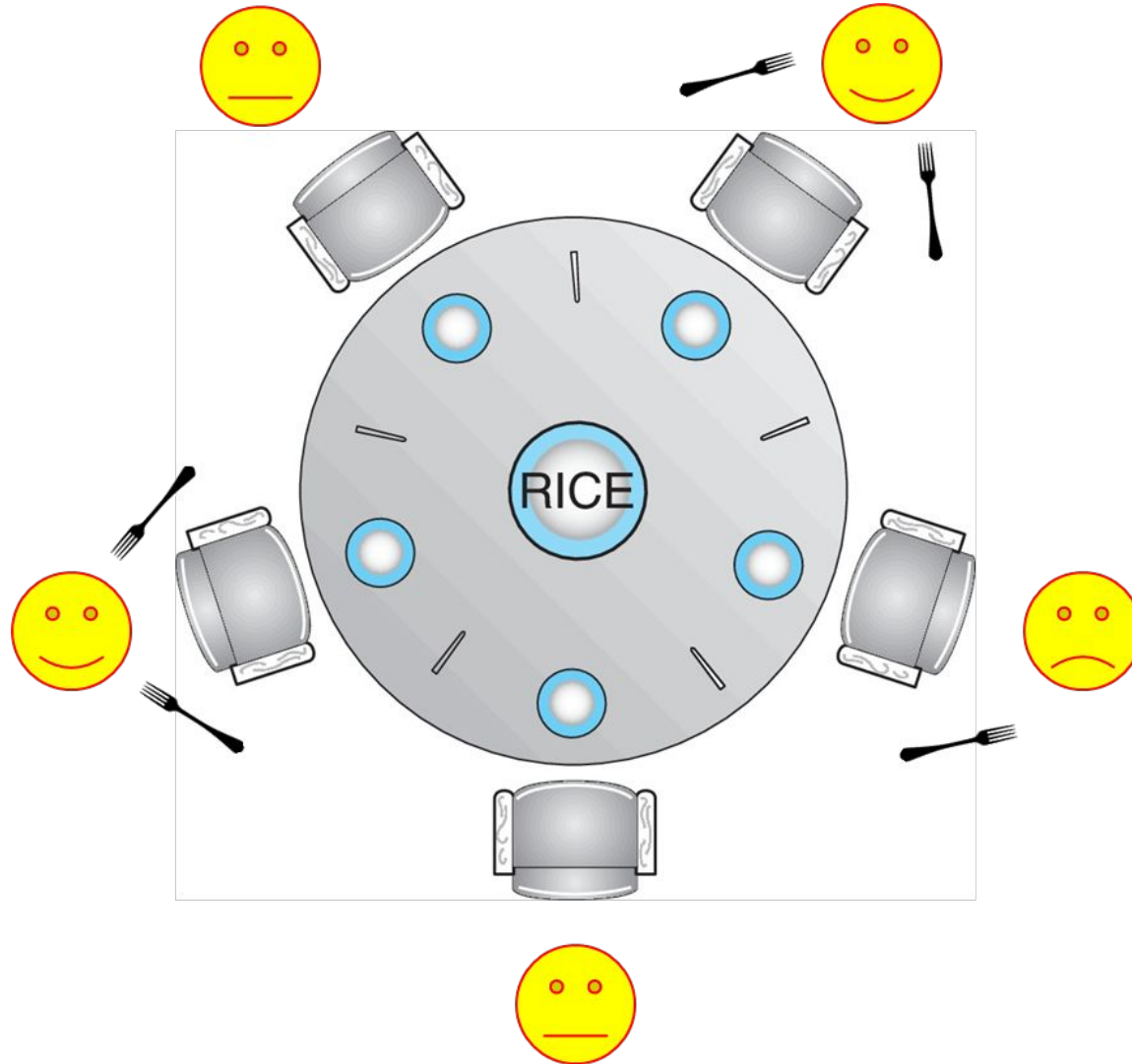
El problema de (dining philosophers problem) fue formulado por Dijkstra en 1965 para representar el problema de sincronización de procesos en un Sistema Operativo.

- Se tienen 5 filósofos sentados en una mesa redonda.
- Entre cada filósofo hay un palillo chino (5 en total)
- Cada filósofo pasa su vida pensando (sin necesitar ningún palillo) o comiendo
- Para comer cada filósofo necesita 2 palillos, el que tiene a su derecha y a su izquierda
- Los filósofos compiten por estos palillos



Problema de los filósofos

Implementación



Basic loop of each philosopher

```
while(1) {  
    think();  
    getforks();  
    eat();  
    putforks();  
}
```

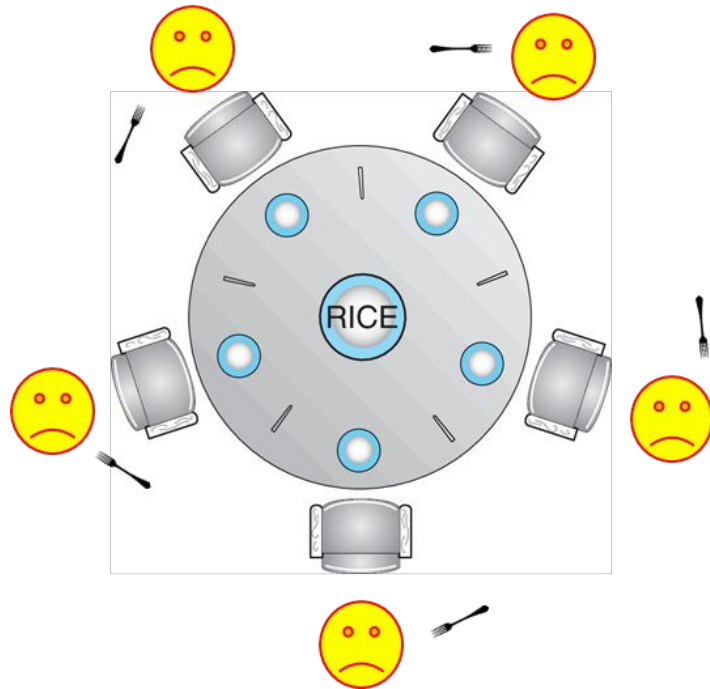
Helper functions (Downey's solutions)

```
// helper functions  
int left(int p) {  
    return p;  
}  
  
int right(int p) {  
    return (p + 1) % 5;  
}
```

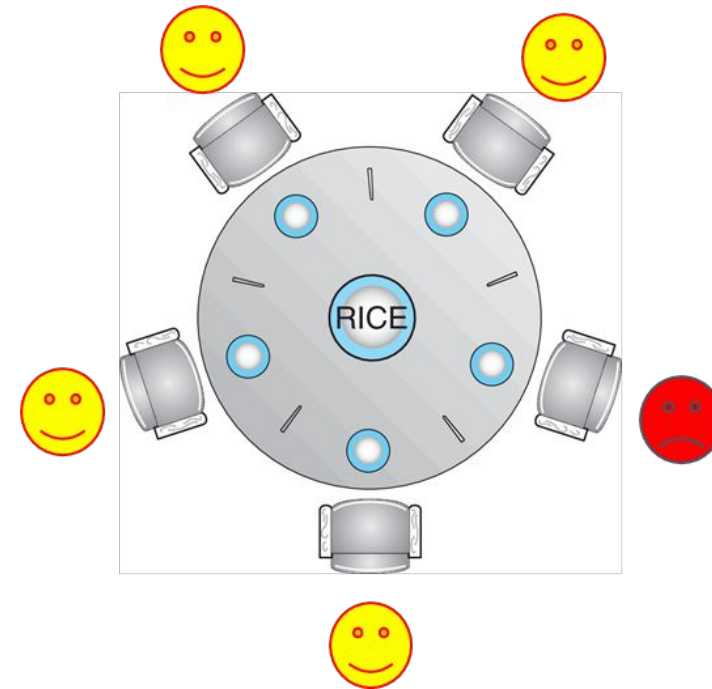
Problema de los filósofos

Objetivos

Objetivo 1: Que no hayan interbloqueos (deadlocks)



Objetivo 2: Que no se quede ningún filósofo sin comer (starvation)



Objetivo 3: Máxima concurrencia

Problema de los filósofos

Implementación

Funciones de utilidad

Filósofo **p** quiere adquirir palillo de la izquierda:
→ Llamar **left(p)**



```
int left(int p) {  
    return p;  
}
```

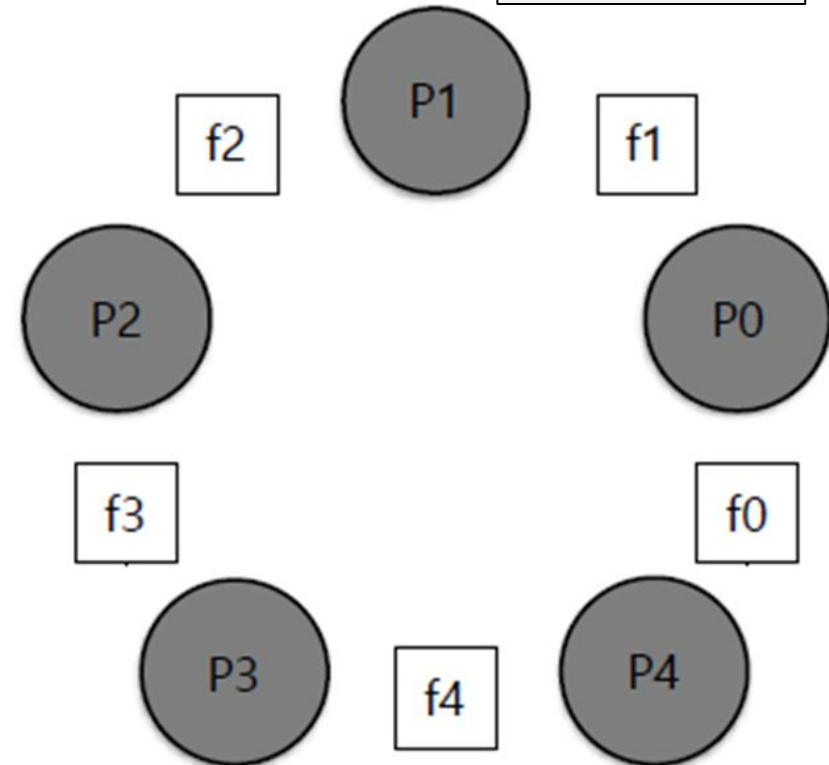
Filósofo **p** quiere adquirir palillo de la derecha:
→ Llamar **right(p)**



```
int right(int p) {  
    return (p + 1) % 5;  
}
```

Basic loop of each philosopher

```
while(1) {  
    think();  
    getforks();  
    eat();  
    putforks();  
}
```



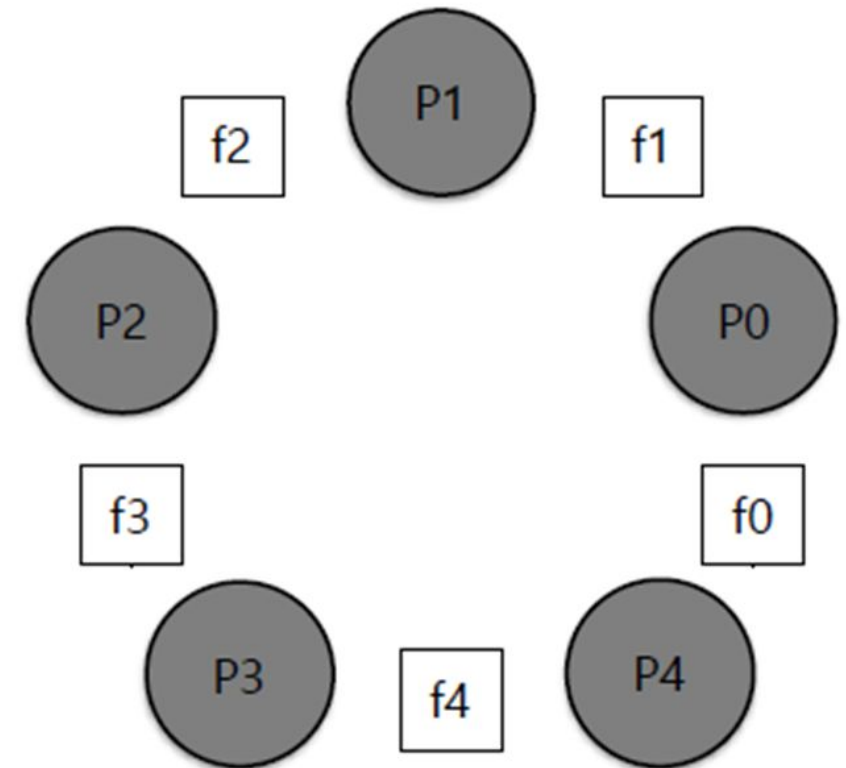
Problema de los filósofos

Implementación

Semáforos

Necesitamos varios semáforos para resolver este problema. Para el caso 5. Uno por cada palillo:

```
sem_t forks[5];
```



Problema de los filósofos

Solución

Solución simple - Primera solución

Implementación inicial del problema

```
// initialized to 1  
sem_t forks[5];
```



```
void philosopher(int i) {  
    while(1) {  
        think();  
        getforks(i);  
        eat();  
        putforks(i);  
    }  
}
```



```
void get_forks(int p) {  
    sem_wait(&forks[left(p)]);  
    sem_wait(&forks[right(p)]);  
}
```



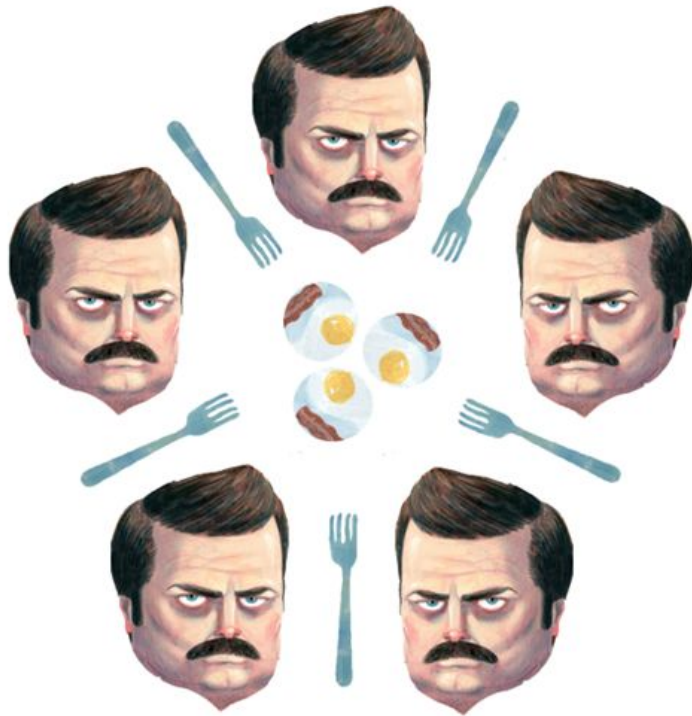
```
void put_forks(int p) {  
    sem_post(&forks[left(p)]);  
    sem_post(&forks[right(p)]);  
}
```

Problema de los filósofos

Solución

Solución simple - Primera solución

Implementación inicial del problema



```
int main(int argc, char *argv[]) {  
    // ...  
    for (int i = 0; i < 5; i++) {  
        sem_init(&forks[i], 0, 1); // initialized to 1  
    }  
    // ...  
}
```

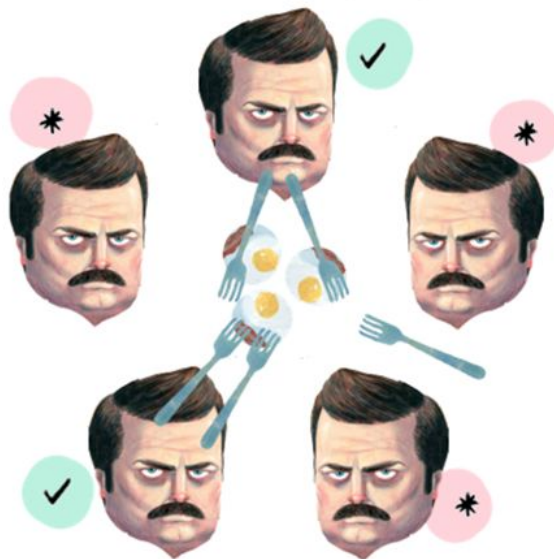
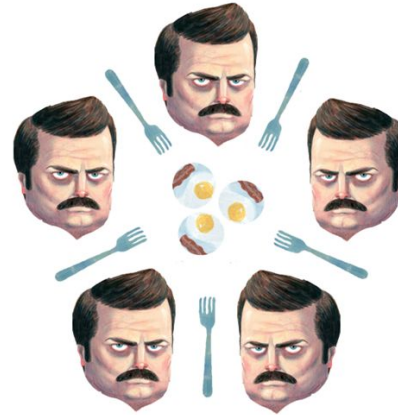

Problema de los filósofos

Solución

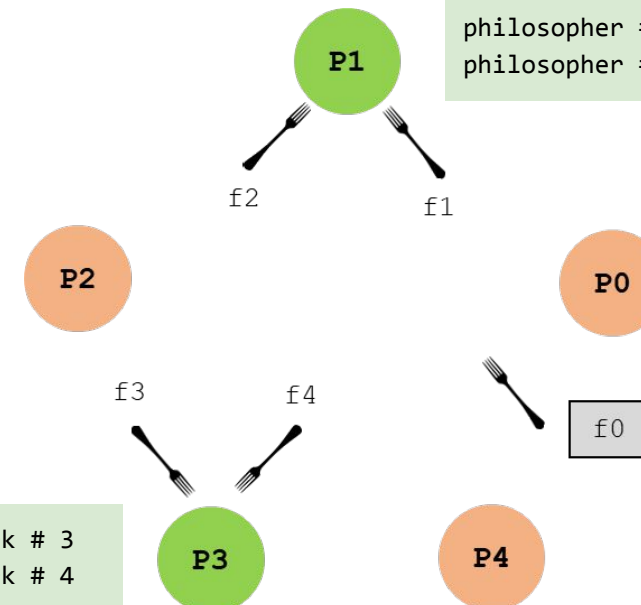
Solución simple - Primera solución

```
int main(int argc, char *argv[]) {
    // ...
    for (int i = 0; i < 5; i++) {
        sem_init(&forks[i], 0, 1); // initialized to 1
    }
    // ...
}
```

```
void philosopher(int i) {
    while(1) {
        think();
        getforks(i);
        eat();
        putforks(i);
    }
}
```



philosopher #3 getfork # 3
philosopher #3 getfork # 4



philosopher #1 getfork # 1
philosopher #1 getfork # 2



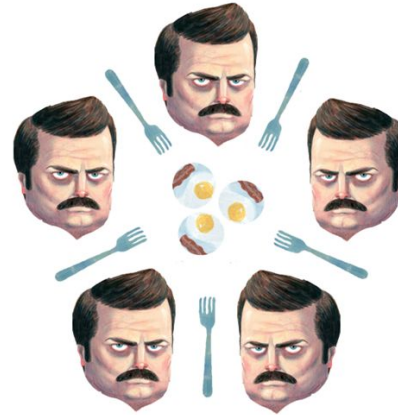
Problema de los filósofos

Solución

Solución simple - Primera solución

```
int main(int argc, char *argv[]) {
    // ...
    for (int i = 0; i < 5; i++) {
        sem_init(&forks[i], 0, 1); // initialized to 1
    }
    // ...
}
```

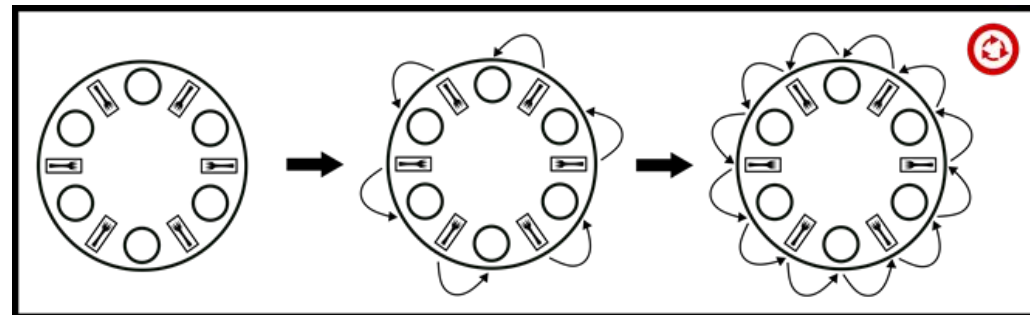
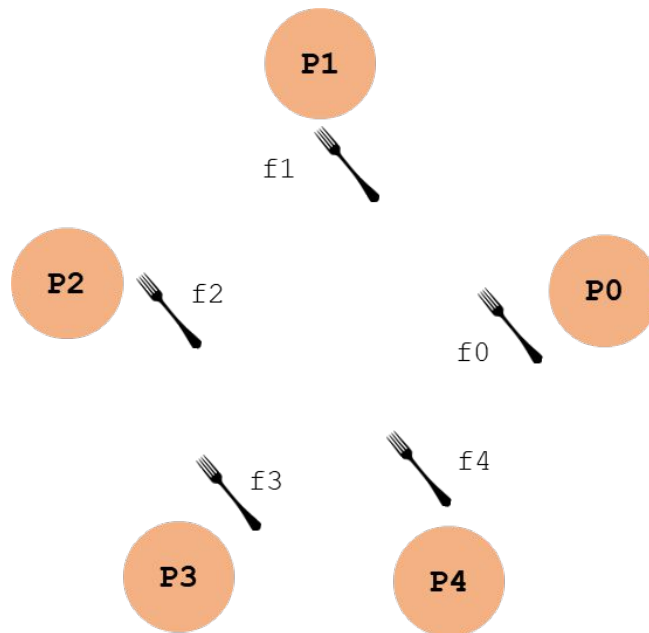
```
void philosopher(int i) {
    while(1) {
        think();
        getforks(i);
        eat();
        putforks(i);
    }
}
```



```
philosopher #0 getfork # 0
philosopher #1 getfork # 1
philosopher #2 getfork # 2
philosopher #3 getfork # 3
philosopher #4 getfork # 4
```



**Interbloqueo
(Deadlock)**



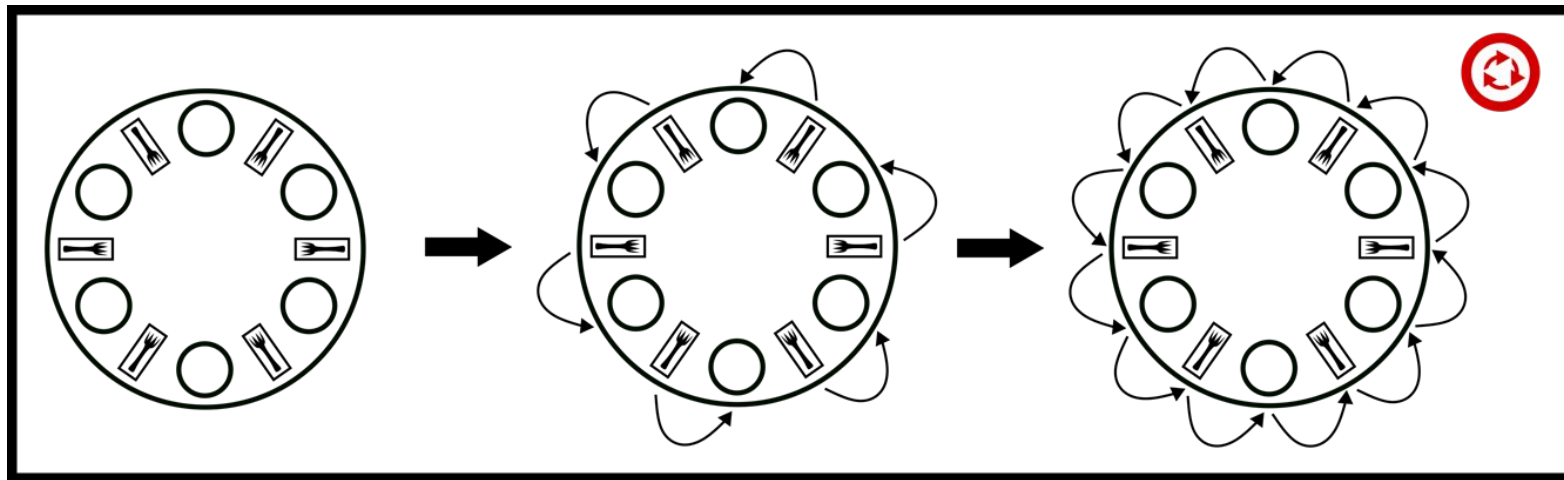
Problema de los filósofos

Solución

Solución simple - Interbloqueo

El interbloqueo se puede dar:

1. Si cada filósofo toma el palillo a su izquierda antes de que cualquier otro filósofo tome el palillo a su derecha.
2. Cada filósofo se quedará bloqueado para siempre, sosteniendo el tenedor y esperando a que otro suelte el palillo.



**Interbloqueo
(Deadlock)**

Problema de los filósofos

Solución

Solución al problema del interbloqueo

Cambiar el orden en que los semáforos son adquiridos

```
// initialized to 1  
sem_t forks[5];
```



```
void philosopher(int i) {  
    while(1) {  
        think();  
        getforks(i);  
        eat();  
        putforks(i);  
    }  
}
```



```
void get_forks(int p) {  
    if (p == 4) {  
        sem_wait(&forks[right(p)]);  
        sem_wait(&forks[left(p)]);  
    }  
    else {  
        sem_wait(&forks[left(p)]);  
        sem_wait(&forks[right(p)]);  
    }  
}
```



```
void put_forks(int p) {  
    sem_post(&forks[left(p)]);  
    sem_post(&forks[right(p)]);  
}
```

Problema de los filósofos

Solución

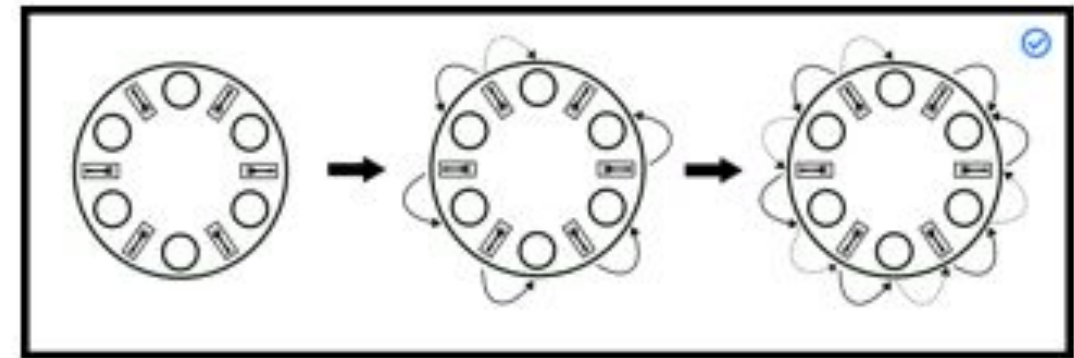
Solución al problema del interbloqueo

Cambiar el orden en que los semáforos son adquiridos:

- Supongamos que para el caso, el último filósofo (4) adquiere los palillos en un orden diferente.
- En este caso no se da la situación en la cual un filósofo que coja un palillo se quede esperando a otro.
- Hay otras soluciones diferentes a esta.
- Existen otros problemas famosos de sincronización como el problema de los fumadores o el problema del barbero dormilón.



```
void get_forks(int p) {  
    if (p == 4) {  
        sem_wait(&forks[right(p)]);  
        sem_wait(&forks[left(p)]);  
    }  
    else {  
        sem_wait(&forks[left(p)]);  
        sem_wait(&forks[right(p)]);  
    }  
}
```



Agenda

- Resumen clase anterior
- Contextualización
- Problema productor consumidor
- Problema de los lectores - Escritores
- Problema de los filósofos
- **Implementación de Zemáforos**
- Conclusiones

Implementación de Zemáforos

Implementación alternativa

A continuación se muestra una implementación alternativa de los **Semaforos** conocida como **Zemaforos** y construida usando Locks y variables de condición.

```
void sem_wait(sem_t *s) {
    s->value--;
    if (s->value < 0) {
        wait;
    }
}

void sem_post(sem_t *s) {
    s->value++;
    if (s->value <= 0) {
        wake one waiting thread;
    }
}
```

```
typedef struct __Zem_t {
    int value;
    pthread_cond_t cond;
    pthread_mutex_t lock;
} Zem_t;

// only one thread can call this
void Zem_init(Zem_t *s, int value) {
    s->value = value;
    Cond_init(&s->cond);
    Mutex_init(&s->lock);
}

void Zem_wait(Zem_t *s) {
    Mutex_lock(&s->lock);
    while (s->value <= 0)
        Cond_wait(&s->cond, &s->lock);
    s->value--;
    Mutex_unlock(&s->lock);
}

void Zem_post(Zem_t *s) {
    Mutex_lock(&s->lock);
    s->value++;
    Cond_signal(&s->cond);
    Mutex_unlock(&s->lock);
}
```

Implementación de Zemáforos

Implementación alternativa

Comentarios sobre la implementación

- El valor del semáforo en esta implementación no representa el número de hilos bloqueados en el semáforo (cuando el valor es negativo).
- Fácil implementación, similar a la implementación actual en linux.

```
void sem_wait(sem_t *s) {  
    s->value--;  
    if (s->value < 0) {  
        wait;  
    }  
}  
  
void sem_post(sem_t *s) {  
    s->value++;  
    if (s->value <= 0) {  
        wake one waiting thread;  
    }  
}
```



```
void Zem_wait(Zem_t *s) {  
    Mutex_lock(&s->lock);  
    while (s->value <= 0)  
        Cond_wait(&s->cond, &s->lock);  
    s->value--;  
    Mutex_unlock(&s->lock);  
}  
  
void Zem_post(Zem_t *s) {  
    Mutex_lock(&s->lock);  
    s->value++;  
    Cond_signal(&s->cond);  
    Mutex_unlock(&s->lock);  
}
```

Agenda

- Resumen clase anterior
- Contextualización
- Problema productor consumidor
- Problema de los lectores - Escritores
- Problema de los filósofos
- Implementación de Zemáforos
- **Conclusiones**

Conclusiones

- Los **semáforos** son equivalentes a **locks + CV** (Variables de condición).
 - Pueden ser usados para exclusión mutua y ordenamiento.
- Los semáforos tienen un estado:
 - La forma como serán inicializados depende de cómo serán usados.
 - **Init sem = 1**: Mutex
 - **Init sem = 0**: Join.
 - **Init sem = N**: Número de recursos disponibles.
 - **Operaciones**:
 - **sem**→**wait()**: Espera hasta que el valor del semáforo sea mayor que 0, entonces decrementa (atómico)
 - **sem**→**post()**: Incrementa el valor del semáforo, entonces despierta a uno de los hilos que se encuentra esperando (Atómico).
- **Problemas de sincronización analizados**:
 - Buffer limitado
 - Escritores y Lectores
 - Cena de filósofos.

Referencias

- <https://pages.cs.wisc.edu/~bart/cs537.html>
- <https://adit.io/posts/2013-05-15-Locks,-Actors,-And-STM-In-Pictures.html>
- https://link.springer.com/chapter/10.1007/978-1-4842-4398-5_5
- <https://preshing.com/20150316/semaphores-are-surprisingly-versatile/>
- <https://jvns.ca/teach-tech-with-cartoons/>
- <https://code-cartoons.com/>
- <https://medium.com/basecs>
- <https://xkcd.com/>
- <https://www.qwantz.com/>
- <https://pbfcomics.com/>
- <http://turnoff.us/>
- <http://turnoff.us/geek/arduino-project/>
- <https://prepinsta.com/operating-systems/critical-section-problem/>
- <https://github.com/capedrazab/os-20191/wiki>
- <https://github.com/jjpulidos/Operating-Systems-UNAL-20191>
- <https://github.com/capedrazab/os-20201/wiki>



Referencias

- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>
- <https://brendangregg.com/linuxperf.html>
- <http://pages.cs.wisc.edu/~harter/537/slides.html>
- <http://www.embeddedlinux.org.cn/rtconforembsys/5107final/LiB0053.html>
- [http://odl.sysworks.biz/disk\\$axpdocsep012/network/dcev30/develop/appdev/Appde179.htm](http://odl.sysworks.biz/disk$axpdocsep012/network/dcev30/develop/appdev/Appde179.htm)
- <https://pages.mtu.edu/~shene/NSF-3/e-Book/MONITOR/monitor-types.html>
- <https://ocw.unican.es/course/view.php?id=240§ion=1>
- <https://omscs.gatech.edu/cs-6200-introduction-operating-systems>
- <https://ruslanspivak.com/lbaws-part1/>
- <https://prepinsta.com/operating-systems/threads/>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/implementing-threads/>
- <https://pages.cs.wisc.edu/~bart/537/lecturenotes/>

Referencias

- <https://github.com/capedrazab/os-20211>
- <https://github.com/pkgamma/cs241-coursebook-clone-fa19/wiki/Synchronization>
- <https://github.com/tiemma/udacity-operating-systems>
- <https://github.com/monorinam/GaTech-CS-6210-Advanced-Operating-Systems>
- <https://github.com/stevenxchung/Introduction-to-Operating-Systems>
- <https://github.com/stevenxchung/Introduction-to-Operating-Systems/tree/master/Lesson%2014%20-%20Synchronization%20Constructs>
- <https://github.com/khanhnamle1994/operating-systems>
- <https://github.com/gauti1311/SystemMonitor>
- https://github.com/bresilla/robond_project_2
- <https://github.com/Maxxquoi/Introduction-to-Operating-Systems>
- <https://github.com/sergioarmgpl/operating-systems-usac-course>
- <https://github.com/MrinmoiHossain/Online-Courses-Learning>
- <https://github.com/PKUFlyingPig/MIT6.S081-2020fall>
- <https://github.com/mishal23/os-lab>



Referencias

- <https://github.com/mishal23/os-simulator>
- <https://github.com/zxc479773533/HUST-OS-Course-Design>
- <https://github.com/mauri870/baking-pi>
- <https://github.com/hyojeonglee/osfall2019>
- <https://github.com/JoyeBright/OSLab>
- <https://github.com/Matrixpecker/VE482-public>
- <https://github.com/MrinmoiHossain/Robot-Operating-System-Udemy>
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes>
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/3-Threads-and-Concurrency.md>
- <http://www.embeddedlinux.org.cn/rtconforembsys/5107final/LiB0053.html>
- <https://levelup.gitconnected.com/producer-consumer-problem-using-condition-variable-in-c-6c4d96efcbbc>
- <http://jakascorner.com/blog/2016/01/condition-variable.html>
- <https://pages.mtu.edu/~shene/NSF-3/e-Book/MONITOR/monitor-types.html>
- <https://github.com/CodeExMachina/ConditionVariable>



Referencias

- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>
- <https://brendangregg.com/linuxperf.html>
- <http://pages.cs.wisc.edu/~harter/537/slides.html>
- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>
- http://ferestrepoca.github.io/paradigmas-de-programacion/paralela/paralela_teorias/index.html
- <https://prepinsta.com/operating-systems/semaphore/>
- <https://www.geeksforgeeks.org/semaphores-in-process-synchronization/>
- <https://preshing.com/20150316/semaphores-are-surprisingly-versatile/>
- <https://prepinsta.com/operating-systems/mutex-vs-semaphore/>
- <https://computationstructures.org/lectures/synchronization/synchronization.html>
- <https://www.slideshare.net/mohammedarif89/semaphores>
- <http://pages.cs.wisc.edu/~harter/537/slides.html>
- <http://www.embeddedlinux.org.cn/rtconforembsys/5107final/LiB0053.html>

Referencias

- <https://pages.mtu.edu/~shene/NSF-3/e-Book/MONITOR/monitor-types.html>
- <https://ocw.unican.es/course/view.php?id=240§ion=1>
- <https://omscs.gatech.edu/cs-6200-introduction-operating-systems>
- <https://ruslanspivak.com/lbaws-part1/>
- <https://prepinsta.com/operating-systems/threads/>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/implementing-threads/>
- <https://www.baeldung.com/cs/readers-writers-problem>

Referencias

- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>
- http://ferestrepoca.github.io/paradigmas-de-programacion/paralela/paralela_teor%C3%ADa/index.html
- <https://prepinsta.com/operating-systems/semaphore/>
- <https://www.geeksforgeeks.org/semaphores-in-process-synchronization/>
- <https://preshing.com/20150316/semaphores-are-surprisingly-versatile/>
- <https://prepinsta.com/operating-systems/mutex-vs-semaphore/>
- <https://computationstructures.org/lectures/synchronization/synchronization.html>
- <https://www.slideshare.net/mohammedarif89/semaphores>
- <http://pages.cs.wisc.edu/~harter/537/slides.html>
- <http://www.embeddedlinux.org.cn/rtconforembsys/5107final/LiB0053.html>
- [http://odl.sysworks.biz/disk\\$axpdocsep012/network/dcev30/develop/appdev/Appdev179.htm](http://odl.sysworks.biz/disk$axpdocsep012/network/dcev30/develop/appdev/Appdev179.htm)
- <https://pages.mtu.edu/~shene/NSF-3/e-Book/MONITOR/monitor-types.html>
- <https://ocw.unican.es/course/view.php?id=240§ion=1>
- <https://omscs.gatech.edu/cs-6200-introduction-operating-systems>



Referencias

- http://nyx.skku.ac.kr/?page_id=2098
- <https://sphof.readthedocs.io/test2.html>
- <https://dev.to/vunderkind/the-dining-philosophers-problem-or-your-introduction-to-thinking-in-computer-science-8ii>
- <http://codebyshaukat.blogspot.com/2014/11/dining-philosophers-problem.html>
- <https://github.com/mtking2/dining-philosophers>
- <https://github.com/mishi-a/Network-OS-Lab-Assignment>
- <https://pages.mtu.edu/~shene/NSF-3/e-Book/MUTEX/TM-example-left-right.html>
- <https://docs.oracle.com/cd/E19205-01/820-0619/gepit/index.html>
- <https://cs241.cs.illinois.edu/coursebook/Deadlock>

UNIVERSIDAD DE ANTIOQUIA

Curso de sistemas Operativos
Clase 18 - Semáforos (Parte 2)